

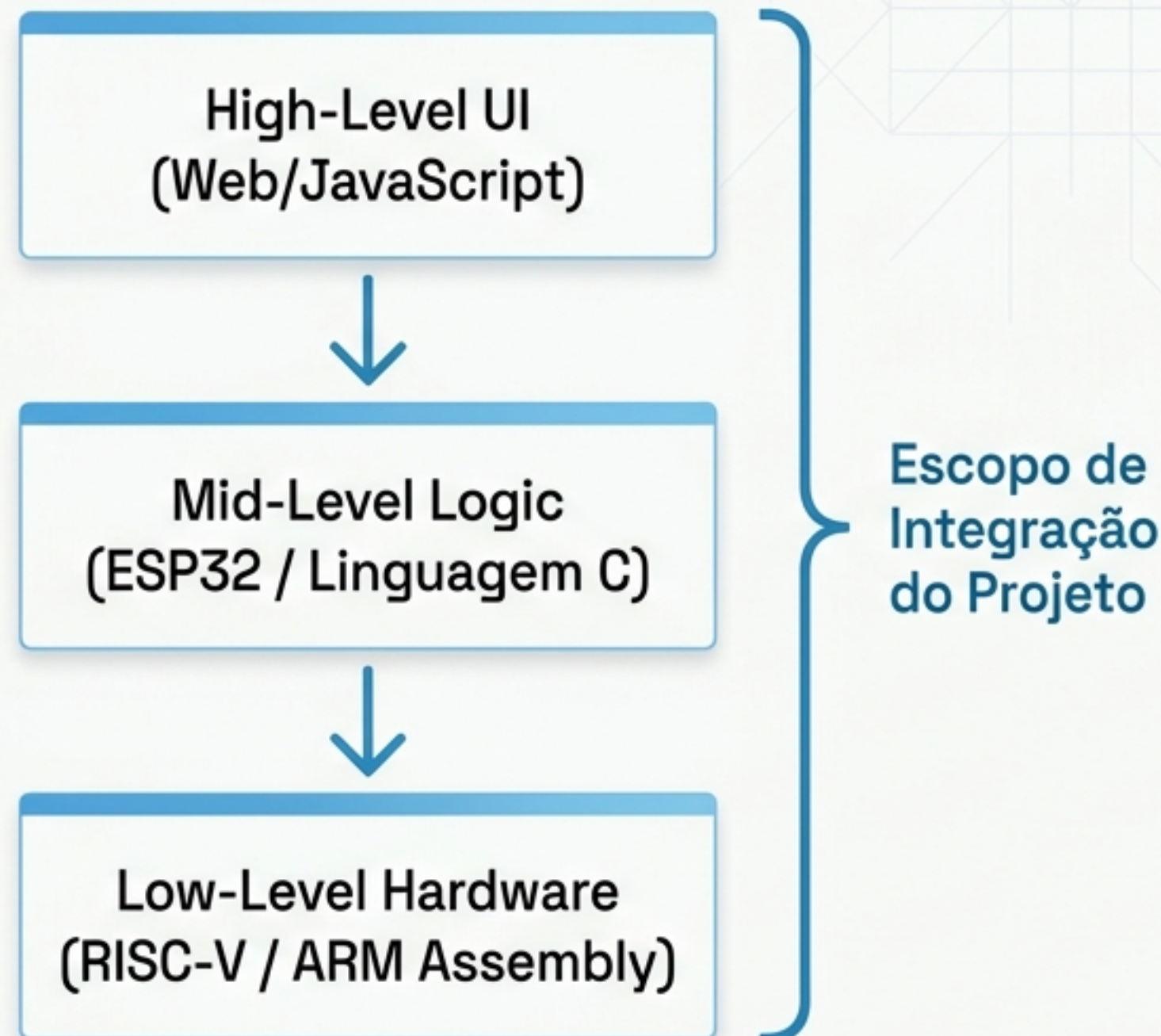
Expansão da Calculadora de 4 Bits no ESP32: Laboratório 04

Disciplina PCS3732 – Laboratório de Processadores

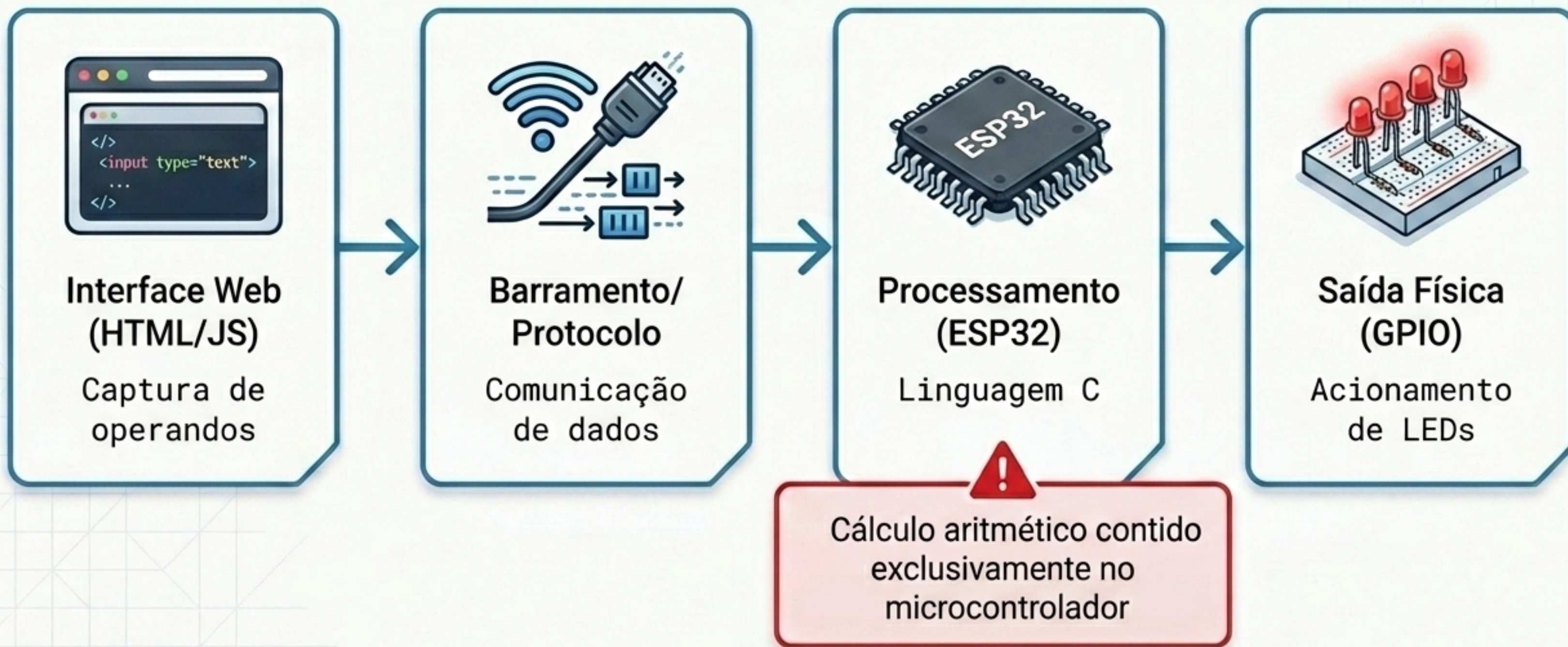
Novas operações de multiplicação e fatorial

Fundamentos da Arquitetura RISC e Níveis de Abstração

- **Transição CISC para RISC:** Redução da complexidade do hardware em favor de instruções simples e otimização de compiladores (Patterson, 2017).
- **Execução eficiente:** Foco em pipelining e ciclo de instrução previsível.
- **Integração do Sistema:** O laboratório exige mapear o fluxo lógico desde o navegador até os registradores físicos.



Arquitetura em Camadas: Da Interface Web ao Hardware Físico



Mapeamento de Requisitos Funcionais e Restrições do Sistema

Requisitos Funcionais (RF)

RF1	Implementar operação de multiplicação.
RF2	Implementar operação de fatorial.
RF3 	Garantir regressão (manter soma e subtração).

Requisitos Não-Funcionais (RNF)

RNF1	Limite inicial de 4 bits para operandos e mais bits considerando escalabilidade.
RNF2 	Implementação lógica estrita na linguagem C.
RNF3	Tempos de execução devem ser mensuráveis.

Matriz de Rastreabilidade e Casos de Teste Planejados

ID	Operação	Entradas	Saída Web	Saída LEDs	Status
T01	Multiplicação	$3 * 2$	6	0110	PENDENTE
T02	Fatorial	$3!$	6	0110	PENDENTE
T03	Soma (Regressão)	$4 + 5$	9	1001	PENDENTE



[Inserir Evidência
Fotográfica/Screenshot
da Interface Web]

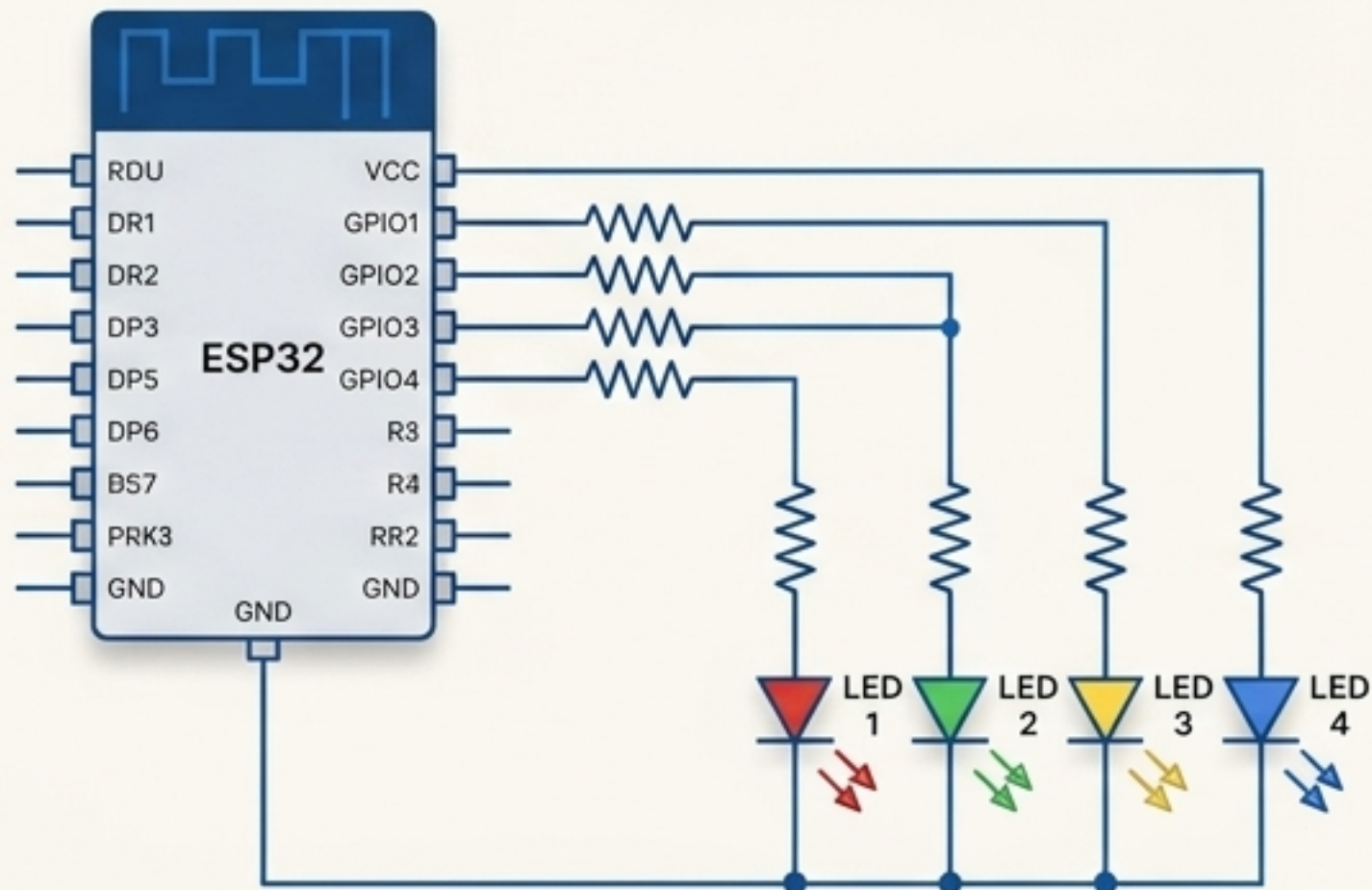
Fluxo de Validação e Método Experimental



A metodologia garante a integração progressiva, isolando falhas de hardware antes da validação da lógica de software.

Validação de Hardware e Testes de Regressão

Diagrama Físico



Console Output

```
> Inicializando GPIO... OK
> Teste LED independente... OK
> Regressão: Soma (2+2=4)... PASS
> Regressão: Subtração (5-2=3)... PASS
> █
```



Confirmação visual de que o hardware base e as rotinas anteriores permanecem intactas.

Implementação Lógica: Multiplicação no Microcontrolador

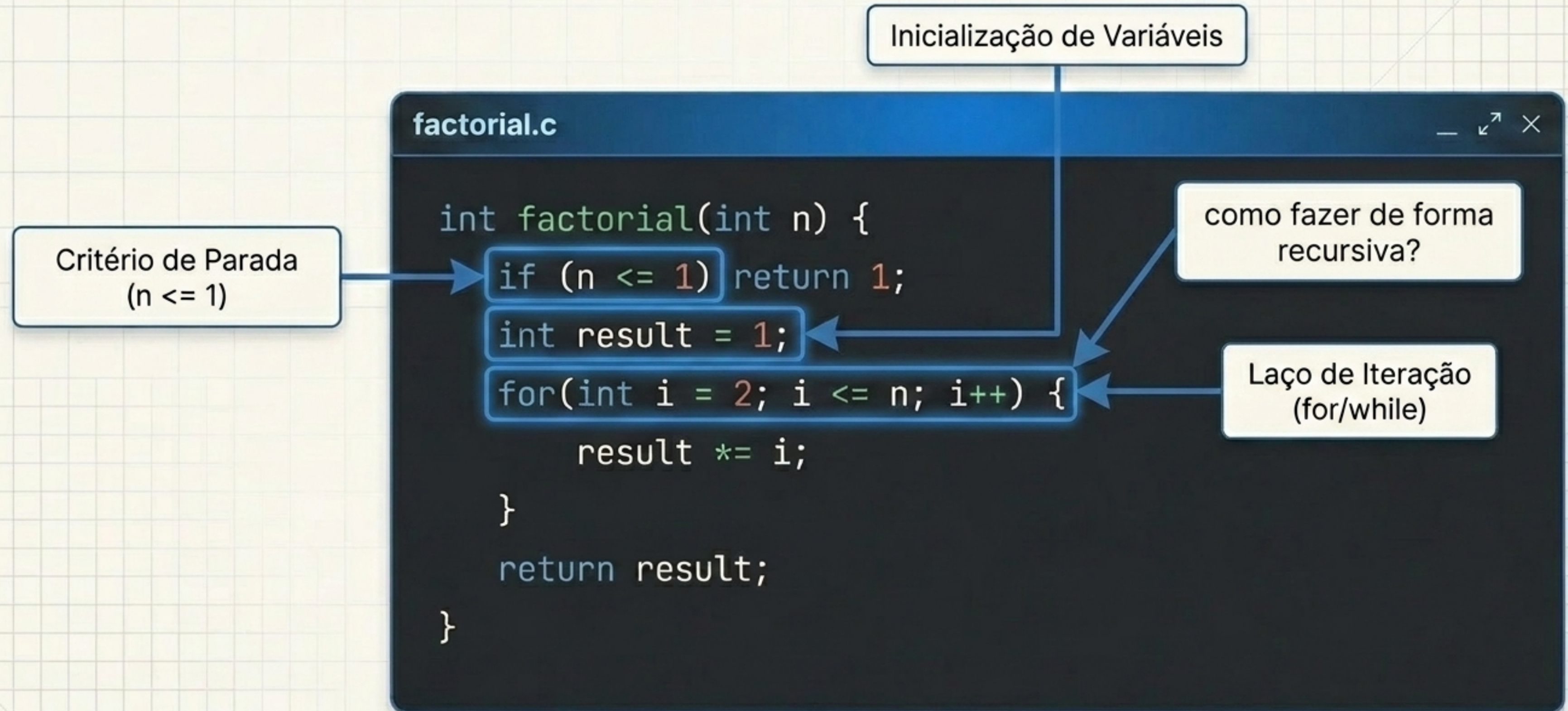
Lógica implementada no backend (ESP32), comprovando a ausência de cálculo no frontend JavaScript.

Gestão do limite de 4 bits
/ Prevenção de overflow

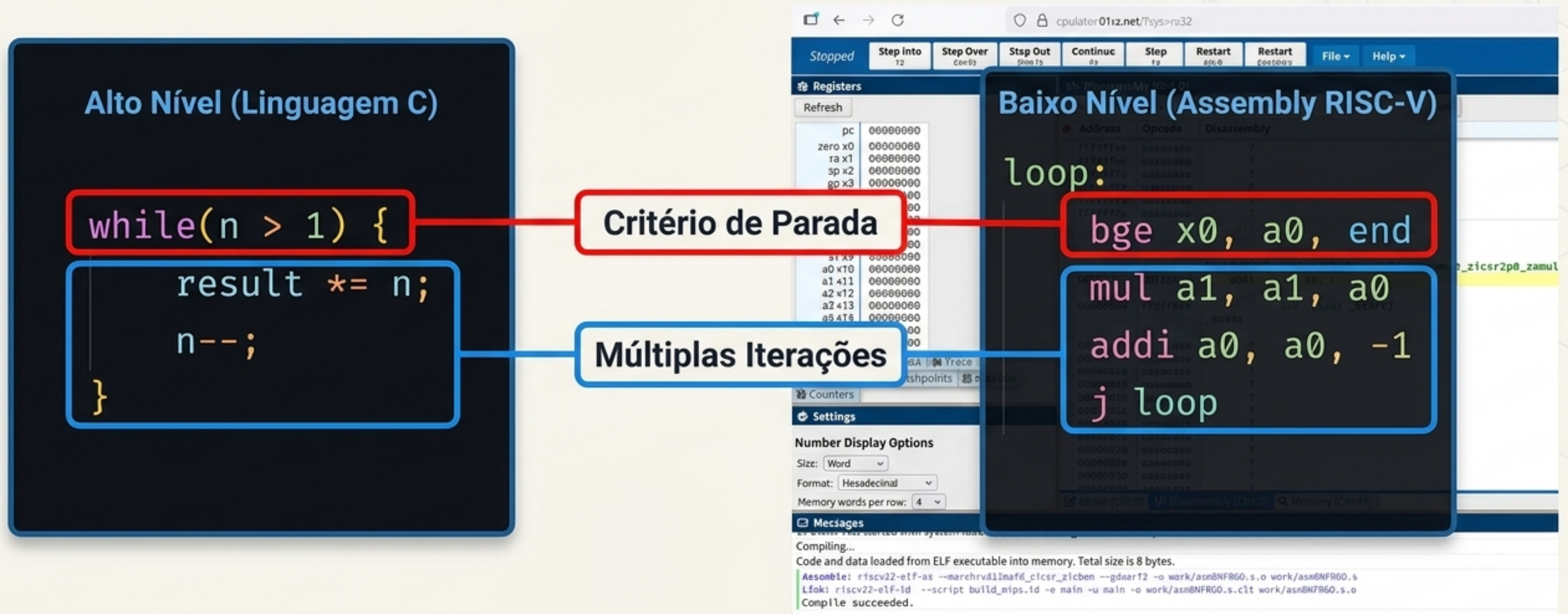
ESP32 - multiply.c

```
int multiply(int a, int b) {  
    int result = 0;  
    for (int i = 0; i < b; i++) {  
        result += a;  
    }  
    return result;  
}
```


Implementação Lógica: Iteração e Controle do Fatorial



Comparativo Estrutural: Abstração em C vs. Assembly RISC-V



Escalabilidade do Sistema e Extensão de Operandos

Original (4 bits)

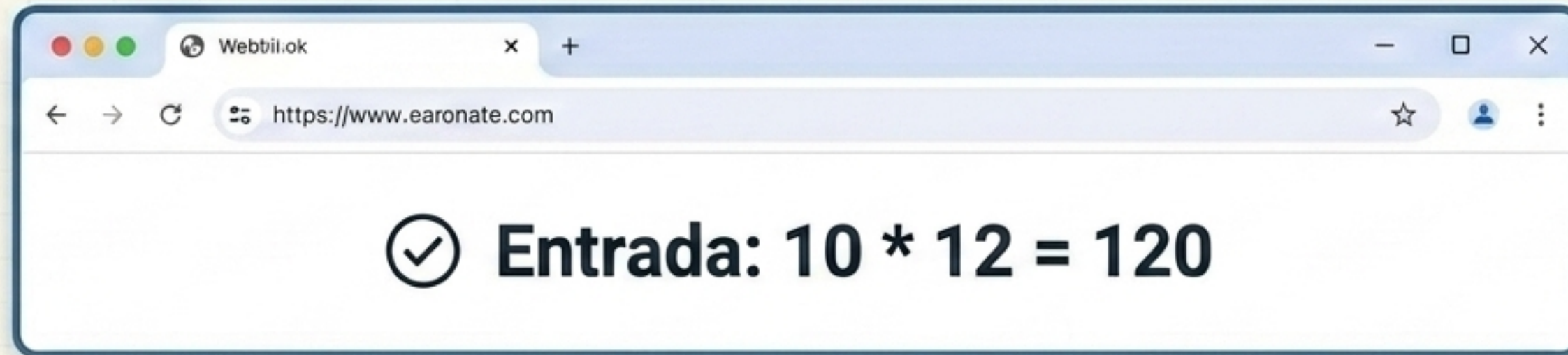


Valor Máximo: 15

Expandido (16 bits)

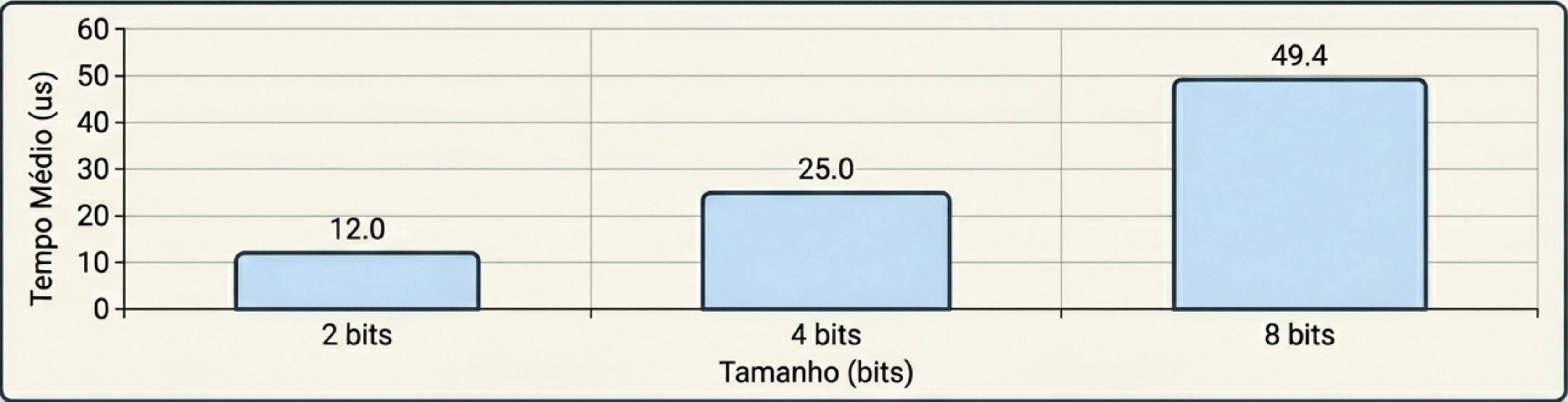


Valor Máximo: 65535



Dashboard: Desempenho da Multiplicação

Operandos	t1 (us)	t2 (us)	t3 (us)	t4 (us)	t5 (us)	Média (us)	Desvio Padrão
2 bits	12	11	12	13	12	12.0	0.7
4 bits	25	24	26	25	25	25.0	0.7
8 bits	48	50	49	51	49	49.4	1.1



Dashboard de Telemetria: Complexidade Temporal do Fatorial

Operandos	t1 (us)	t2 (us)	t3 (us)	t4 (us)	t5 (us)	Média (us)	Desvio Padrão
N = 3	15	14	15	16	15	15.0	0.7
N = 5	35	34	36	35	35	35.0	0.7
N = 8	85	88	86	89	87	87.0	1.5



Como avaliar o resultado esperado considerando a análise de complexidade de algoritmos?

Qual a **diferença** entre a complexidade de algoritmos e a medição temporal de uma implementação?

Desafio Adicional: Integração da Operação de Divisão

```
1 int divide(int a, int b) {  
2  
3     if (b == 0) return -1;  
4     return a / b;  
5 }
```



Tratamento de Exceção: Divisão
por Zero

Operando	Média (us)
4 bits	30.2
8 bits	55.4
16 bits	90.1

10 / 2 = 5

Lições Aprendidas

Desafios de Hardware

Depuração de conexões GPIO físicas e estabilização dos sinais lógicos na protoboard.

Desafios de Software

Tratamento sistemático de overflow nas variáveis maiores e alinhamento da comunicação entre o frontend (JS) e o microcontrolador (C).

Validação Metodológica

O método experimental assegurou rastreabilidade de ponta a ponta, desde a arquitetura de blocos até a detalhada de performance ancorada na ISA RISC-V.

